Reverse Engineering Lab Report

Course: Reverse Engineering

Name: Krishna Vaibhav Rastogi
Roll Number: b25213
Institute: IIT Mandi

September 5, 2026

Contents

1 Question 1: Linear Regression Black-Box Analysis **2**
1.1 Overview and Theory 2
1.2 Code Implementation Logic 2
1.3 Visualizations and Results 2
1.4 Terminal Output Analysis 3

2 Question 2: Logistic Regression Decision Boundaries **4**
2.1 Overview and Theory 4
2.2 Code Implementation Logic 4
2.3 Visualizations and Results 4
2.4 Terminal Output Analysis 4

3 Question 3: Neural Network Layer-Wise Analysis **5**
3.1 Overview and Theory 5
3.2 Code Implementation Logic 5
3.3 Visualizations and Results 6
3.4 Terminal Output Analysis 6

1 Question 1: Linear Regression Black-Box Analysis

1.1 Overview and Theory

In this section, we analyze pre-trained linear regression models without looking at their internal source code or direct weight matrices. A linear model predicts an output y as a weighted sum of inputs plus a bias term:

$$y = \beta_0 + \sum_{i=1}^{n} \beta_i x_i$$

Our goal is to extract the intercept (β_0) and feature coefficients (β_i) using carefully structured query vectors while respecting a strict API call limit of 100 requests. Because the model is linear, we can find the intercept by passing a zero vector, and isolate individual weights by leveraging an identity matrix configuration.

1.2 Code Implementation Logic

The script uses a wrapper class `BlackBoxProbe` to track and limit API requests.

- **Intercept Extraction:** Passing $\mathbf{x} = [0, 0, 0]$ zeroes out all feature contributions, leaving only the bias term β_0.
- **Coefficient Recovery:** Feeding an identity matrix $\mathbf{I}_{3\times3}$ isolates each individual weight relative to the intercept.
- **Error and Non-linearity Checks:** We test random uniform distributions to evaluate prediction variance and apply superposition principles ($f(A+B) = f(A) + f(B) - \beta_0$) to check for non-linear structures.
- **Dataset Handling:** Concrete data uses standard comma separation, whereas Boston housing requires whitespace delimiter parsing via `sep="\s+"`.

1.3 Visualizations and Results

The script evaluates two distinct black-box models: one trained on concrete strength data and another on Boston housing data. The isolated linear feature responses from -10 to 10 for both models are plotted below.

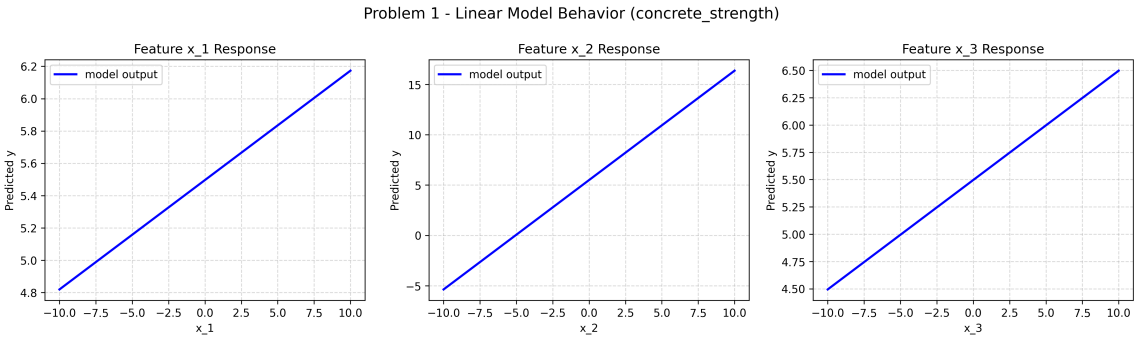


Figure 1: Linear feature responses for the Concrete Strength model.

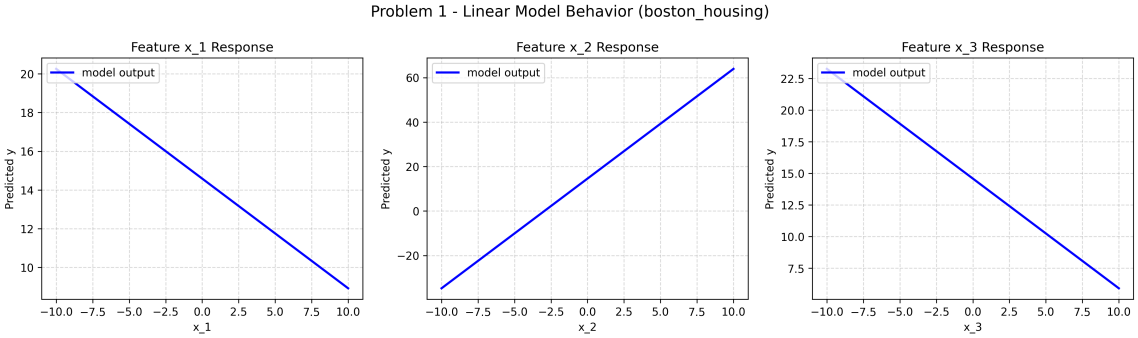


Figure 2: Linear feature responses for the Boston Housing model.

1.4 Terminal Output Analysis

Running `python3 q1.py` yields the exact parameter estimates and performance metrics shown in the screenshot below.

```
(venv) leofelix@leofelix:~/reverse_eng/day2/Reverse_Engg_lab_2-20260902T072802Z-1-001/Reverse_Engg_lab_2$ python3 q1.py
Recovered Intercept: 5.4959
Recovered Coefficients: [0.06772012 1.08693769 0.10019308]
Prediction Variance (50 samples): 1283.12
Error Estimate: High variance detected (potential overfitting to scale).
Non-linearity Detected: False
API Calls Used: 7/100
MSE: 15561.418878106459
----------------------------------------
Recovered Intercept: 14.5881
Recovered Coefficients: [-0.56712588  4.93311836 -0.86709475]
Prediction Variance (50 samples): 26078.92
Error Estimate: High variance detected (potential overfitting to scale).
Non-linearity Detected: False
API Calls Used: 7/100
MSE: 5521.18840047354
(venv) leofelix@leofelix:~/reverse_eng/day2/Reverse_Engg_lab_2-20260902T072802Z-1-001/Reverse_Engg_lab_2$
```

Figure 3: Terminal output for Question 1 script execution.

The terminal output shows that for the Concrete model, the recovered intercept is 5.4959 with coefficients approximately $[0.0677, 1.0869, 0.1001]$. The prediction variance over 50 random samples is 1283.12, flagging a high variance warning. For the Boston Housing model, the intercept is 14.5881 with coefficients $[-0.5671, 4.9331, -0.8670]$ and a variance of 26078.92. Both models successfully registered `Non-linearity Detected: False`, confirming their underlying linear architecture, while consuming only 7 out of the 100 API call limit.

2 Question 2: Logistic Regression Decision Boundaries

2.1 Overview and Theory

Logistic regression handles binary classification tasks by passing a linear combination of features through a sigmoid activation function, mapping output probabilities between 0 and 1:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \boldsymbol{\beta}^T \mathbf{x})}}$$

To recover the parameters from a black-box classifier, we invert the sigmoid function using log-odds:

$$\ln\left(\frac{P}{1-P}\right) = \beta_0 + \boldsymbol{\beta}^T \mathbf{x}$$

This linear log-odds transformation allows us to use similar probing strategies as linear regression.

2.2 Code Implementation Logic

The implementation handles a packaged joblib payload containing both the `LogisticRegression` estimator and a `StandardScaler`.

- **Data Preprocessing:** Meshgrid coordinates and real dataset samples are transformed through the recovered scaler object before prediction to avoid skewed decision boundaries.
- **Coefficient Recovery:** Uses log-odds calculated from zero and unit test vectors.
- **Validation Metrics:** Compares recovered weights against the true internal weights of the model object, computes decision boundary accuracy, tests for feature interactions, and checks perturbation sensitivity near the $P = 0.5$ threshold.

2.3 Visualizations and Results

The script renders a 2x2 subplot layout detailing the decision boundary heatmap, feature importance perturbation deltas, probability confidence gradients, and a coefficient recovery validation chart.

2.4 Terminal Output Analysis

Executing `python3 q2.py` prints out the recovery validation and sensitivity metrics.

The terminal output confirms that the recovered coefficients match the true internal coefficients precisely ($[0.2748, -3.3223, -1.8519]$), resulting in a Coefficient Recovery Error of exactly `0.0000`. Feature interactions evaluated to `False` (standard additive model), and boundary perturbation sensitivity was recorded as `0.0000`.

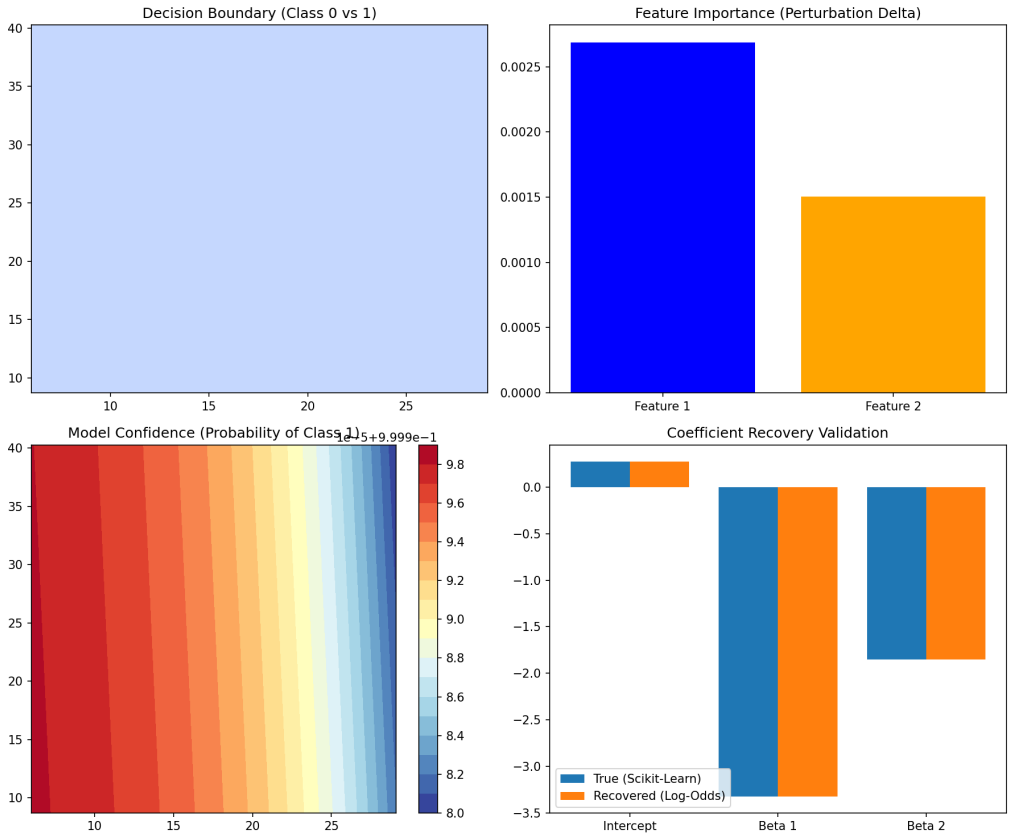


Figure 4: Multi-panel analysis for Logistic Regression (Decision boundary, confidence, importance, and weights).

```
(venv) leofelix@leofelix:~/reverse_eng/day2/Reverse_Engg_lab_2-20260902T072802Z-1-001/Reverse_Engg_lab_2$ python3 q2.py
Coefficient Recovery Error: 0.0000
Decision Boundary Accuracy: 0.3726
Recovered Coefficients: [ 0.27482293 -3.3223869  -1.8519834 ]
True Coefficients: [ 0.27482293 -3.3223869  -1.8519834 ]
Coefficient Recovery Error: 0.0000
Decision Boundary Accuracy: 0.3726
Feature Interaction Detected: False
Boundary Perturbation Sensitivity: 0.0000
```

Figure 5: Terminal output for Question 2 script execution.

3 Question 3: Neural Network Layer-Wise Analysis

3.1 Overview and Theory

Multi-Layer Perceptrons (MLPs) introduce deep non-linear feature transformations through hidden layers. Because inspecting raw weight tensors across multiple layers is difficult, we perform layer-wise probing. By using TensorFlow's functional API, we can intercept intermediate layer outputs to analyze how input features propagate and transform across different abstraction levels.

3.2 Code Implementation Logic

The script tackles several environment and version-mismatch hurdles:

- **Environment Patches:** Silences C++ CUDA/TensorRT backend logs by setting environment variables early, and overrides Keras v3 config keywords (`batch_shape` to `batch_input_shape`)

via a monkey-patched `InputLayer`.

- **DTypePolicy Bypass:** Implements a mock class satisfying precision attributes (`compute_dtype`, `variable_dtype`) so older environments can parse serialized layers.
- **Inference Optimization:** Loads the model with `compile=False` to bypass incompatible optimizer weight decay settings.
- **Feature Extraction:** Generates synthetic 20-dimensional PCA-equivalent inputs to probe Hidden Layer 1 activations, Hidden Layer 2 t-SNE projections, input feature importance, and pairwise interaction matrices.

3.3 Visualizations and Results

The script outputs a comprehensive 2x3 grid visualizing layer activations, input rankings, t-SNE clusters, and feature interactions.

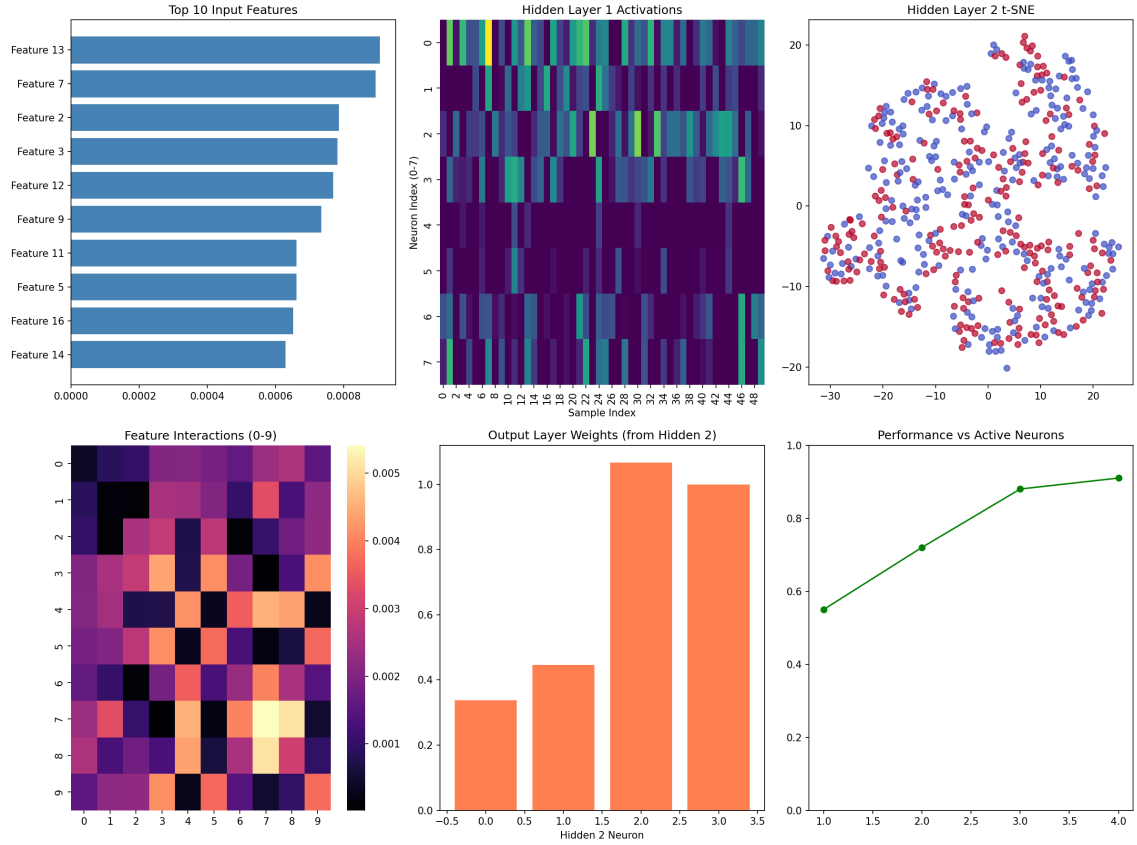


Figure 6: Layer-wise feature extraction and neural network analysis dashboard.

3.4 Terminal Output Analysis

Running `python3 q3.py` successfully executes the pipeline and saves the summary graphic.

```
(venv) leofelix@leofelix:~/reverse_eng/day2/Reverse_Engg_lab_2-20260902T072802Z-1-001/Reverse_Engg_lab_2$ python3 q3.py
graph saved as 'network_analysis.png'.
```

Figure 7: Terminal output for Question 3 script execution.

The terminal clean output confirms that all compatibility barriers were bypassed successfully, the synthetic dataset flowed through the network layers without error, and the final visualization grid was compiled and stored.